In our example above, it means that of the two calls to `set`, while the first is executed, the second is left waiting.

As you can see, this is a powerful mechanism, but it's also dangerous, because failing to release a lock can leave all other calls hanging forever (*deadlock*) and even just exceeding with the use of locks can degrade performance sensibly, introducing unnecessary long delays.

Locks basically move execution from parallel to synchronous (hence the term *synchronization*) by using waiting queues to regulate access to a shared resource.

Explaining in-depth how lock works and how deadlocks can happen and can be prevented usually takes a couple of chapters of a book on operating systems, and so it's out of our scope: we encourage interested readers to go on and read more on the subject[30], since it's becoming more and more important for modern programming and web applications, but also compute-intensive application that runs in parallel on GPUs.

Now, what does reentrant mean? As the word suggest, with a reentrant lock a thread can enter (or lock) a resource more than once, and every time it does, a counter is incremented by one, while when it unlocks the resource, the counter is decremented by one, and the lock is actually released when this counter gets to zero. Why is it important to have a reentrant lock? Because if the same thread might try to lock the same resource more than once, we could get into a deadlock.